PRODUCT REQUIREMENTS DOCUMENT

Sticker Sheet Automation & Print-and-Cut Production Application

Client	Jona C
Prepared by	FreelanceComm
Application	Standalone Desktop Application
Frontend	React
Backend	Firebase
Development Structure	2 Phases
Initial Timeline	25 Days
Initial Budget	$150
Future Expansion	Licensing + Monthly/Yearly Subscription

Document purpose: define the complete Phase 1 frontend/desktop production application and Phase 2 Firebase backend architecture agreed from the client conversation. Subscription and licensing are explicitly treated as future expansion unless separately approved.

1. Product Vision

Build a professional standalone desktop application that automates sticker-sheet preparation for print-and-cut production. The application replaces repetitive manual work currently performed in software such as Illustrator/CorelDRAW.

Core workflow:

```
Drag Artwork → Configure Sticker → Generate Contour → Select Media → Auto Arrange/Nest → Add Registration Marks → Preview → Export PDF/SVG
```

The application must be simple enough for a production operator while providing control over all important production parameters.

2. Technology Stack

Frontend

- React
- JavaScript/TypeScript
- Modern component architecture
- Desktop wrapper/runtime such as Electron or Tauri
- Local file-system access
- Local production processing

Backend

- Firebase Authentication
- Firebase Firestore
- Firebase Storage
- Firebase Security Rules
- Firebase configuration/environment variables

Version Control

- GitHub

3. Critical Architecture Principle

Separate the production engine from the cloud backend.

Local Production Engine

- Artwork import
- Image/vector processing
- Sticker sizing
- Contour generation
- Contour offset
- Smoothing
- Duplication
- Nesting
- Layout

- Registration marks
- Preview
- PDF generation
- SVG generation

The user should not need an internet connection simply to prepare and export stickers.

Firebase Backend

- Authentication
- User profiles
- Application settings
- Project metadata
- Cloud project storage where required
- Future licensing
- Subscription information
- Usage information

PHASE 1 — FRONTEND + DESKTOP PRODUCTION APPLICATION

Objective: Build the complete functional desktop application interface and local sticker-production engine. Phase 1 must produce a working desktop application where the user can take artwork through the complete production workflow without requiring Firebase for core processing.

4. Desktop Application Shell

- React application packaged as a standalone desktop application
- Application launch screen
- Main production workspace
- Desktop window
- File-system access
- Import/export dialogs
- Application menu where appropriate
- Error handling
- Loading states
- Production status indicators

5. Main Application Layout

- **Header:** Application logo/name, New Project, Open Project, Save Project, Undo, Redo, Zoom controls, Preview, Export
- **Left Sidebar:** Import Artwork, Artwork list, Sticker settings, Contour settings, Media settings, Layout settings, Registration settings
- **Center Canvas:** Main production preview
- **Right Sidebar:** Contextual settings and properties
- **Bottom Status Bar:** Sticker quantity, media size, sticker size, current zoom, layout status, export status

6. Artwork Upload

- PNG
- JPG
- JPEG
- SVG
- PDF
- Drag-and-drop directly into the application
- File validation before importing

7. Artwork Management

- Thumbnail
- Filename
- File type
- Dimensions
- Quantity
- Selected/unselected state
- Select artwork
- Remove artwork
- Replace artwork
- Duplicate artwork where applicable

8. Sticker Configuration

- Width
- Height
- Inches and millimeters
- Quantity
- Rotation control
- At minimum 0° and 90° rotation

9. Contour Generation

Automatically generate a vector contour around the artwork using the flow:

```
Artwork → Boundary Detection → Contour → Offset → Smoothing → Vector Cut Path
```

The contour must be an actual vector path.

10. Contour Controls

- Contour enabled/disabled
- Offset
- Smoothing
- Regenerate contour

```
Example: Contour Offset: 0.10" | Smoothing: 50%
```

Changes should update the preview.

11. Media Configuration

- Sheet: width and height
- Roll: roll width and production length
- Portrait and landscape orientation
- Arbitrary custom dimensions

12. Margins

- Top
- Bottom
- Left
- Right

```
Example: Top 0.25" | Bottom 0.25" | Left 0.25" | Right 0.25"
```

The nesting engine must respect these boundaries.

13. Sticker Spacing

- Horizontal spacing
- Vertical spacing

The spacing must be respected during automatic nesting.

14. Automatic Arrangement

Automatically arrange stickers according to sticker dimensions, quantity, media dimensions, margins, spacing, and rotation. The system should maximize practical material usage.

15. Nesting Engine

- Reduce wasted media
- Fit the maximum practical number of stickers
- Respect spacing
- Respect margins
- Respect rotation
- Keep stickers inside the production area

If the requested quantity cannot fit, display a clear warning.

16. Registration Marks

Automatically generate registration marks for print-and-cut workflows. The primary target is the Graphtec FC8600.

- Appear in preview
- Position outside sticker artwork
- Respect media boundaries
- Included in exported files

17. Registration Mark Settings

- Mark size
- Mark spacing
- Mark placement

- Registration area
- Number/position of marks

The design should allow additional cutter configurations in the future.

18. Production Layers

- **Artwork:** Original artwork
- **Cut Contour:** Vector contour paths
- **Path Line:** Production path-line information
- **Registration Marks:** Registration marks for the print/cut workflow

Maintain these as separate logical layers internally and preserve them during export wherever the target format supports layers.

19. Live Production Preview

- Media boundaries
- Artwork
- Sticker copies
- Contour paths
- Registration marks
- Margins
- Spacing

Changes should update the preview.

20. Canvas Controls

- Zoom in
- Zoom out
- Fit to screen
- Reset zoom
- Pan
- Optional mouse-wheel zoom

The preview should remain smooth when displaying many sticker instances.

21. Production Information

```
Sticker Size: 3" × 2"
Quantity: 100
Media: 24" × 36"
Spacing: 0.15"
Margins: 0.25"
Contour: 0.10"
Placed: 96 / 100
```

22. PDF Export

- Production-ready PDF
- Correct physical dimensions
- Artwork
- Contour paths

- Path lines
- Registration marks
- Correct layout
- No unexpected scaling

23. SVG Export

- Artwork
- Vector contour paths
- Registration marks
- Correct physical dimensions
- Production layers where supported
- Appropriate physical/document units

24. Export Validation

- Media dimensions
- Sticker dimensions
- Quantity
- Contour existence
- Valid vector paths
- Registration marks
- Layout boundaries
- Object positions

Display meaningful errors instead of generating an invalid production file.

25. Local Project Management

- New Project
- Open Project
- Save Project
- Save As

Project state should include project name, artwork, sticker size, quantity, contour settings, media settings, spacing, margins, rotation, registration settings, and layout. The project format should be designed so it can later synchronize with Firebase.

26. Phase 1 User Journey

```
Launch Application
↓
Create New Project
↓
Drag Artwork
↓
Select Artwork
↓
Enter Sticker Width/Height
↓
Enter Quantity
↓
Generate Contour
```

```
↓
Set Offset/Smoothing
↓
Select Sheet/Roll
↓
Set Media Dimensions
↓
Set Margins
↓
Set Spacing
↓
Configure Rotation
↓
Auto Arrange/Nest
↓
Generate Registration Marks
↓
Preview
↓
Validate
↓
Export PDF/SVG
```

PHASE 2 — FIREBASE BACKEND

Objective: Add Firebase-based cloud infrastructure without making Firebase a requirement for the local production engine. Establish the foundation for future licensing/subscription functionality.

27. Firebase Authentication

- Email/password authentication
- Google authentication if required later
- User sessions

28. User Profiles

```
users/{userId}
name
email
createdAt
updatedAt
accountStatus
```

29. Firestore Database

Potential collections:

```
users
projects
licenses
subscriptions
devices
usage
settings
```

Only collections required for the agreed Phase 2 implementation should be activated.

30. Cloud Project Storage

Design Firebase Storage integration for artwork/project files.

```
users/{userId}/projects/{projectId}/
```

- Project files
- Uploaded artwork
- Generated project assets

Large production files should not be unnecessarily stored in Firestore.

31. Cloud Project Metadata

```
projectId
userId
projectName
createdAt
updatedAt
mediaSettings
stickerSettings
layoutSettings
contourSettings
registrationSettings
```

32. Project Synchronization

```
Desktop Application
↓
Firebase
```

```
↓
Cloud Project
```

- Save project to cloud
- Load project from cloud
- Update project
- Delete project

If full synchronization is outside the agreed Phase 2 scope, implement the backend architecture and service layer without overextending delivery.

33. Firebase Security

Implement Firebase Security Rules so users can only access their own profiles, projects, files, and settings. Cross-user private project access must be prevented.

34. Firebase Storage Security

```
User A → User A files
User B → User B files
```

Storage rules must prevent unauthorized cross-user access.

35. Backend Service Architecture

```
React Components
↓
Application Services
↓
Firebase Services
↓
Firestore / Storage / Auth
```

Recommended service boundaries include authService, projectService, storageService, userService, settingsService, licenseService, and subscriptionService.

36. Future Licensing Architecture

Licensing is a future feature discussed with the client. Phase 2 should be structured so it can eventually support:

```
licenseId
userId
licenseKey
status
createdAt
expiresAt
```

37. Future Subscription Plans

- Monthly subscription
- Yearly subscription

Potential states: TRIAL, ACTIVE, PAST_DUE, CANCELLED, EXPIRED, SUSPENDED. These are future-ready states and should not block the current production application.

38. Future Payment Integration

```
User
↓
Select Monthly/Yearly Plan
↓
Payment Gateway
```

```
↓
Successful Payment
↓
Subscription Created
↓
License Activated
↓
Desktop App Validates License
```

Payment credentials must never be hardcoded into the React application.

39. Future Desktop License Validation

```
Desktop App
↓
Authentication
↓
License Validation
↓
Firebase/Backend
↓
License Status
```

License enforcement should be introduced only when the licensing system is officially added.

40. GitHub Development Workflow

Maintain the project in GitHub with structured source code, meaningful commits, README documentation, build instructions, and separate configuration for development and production environments.

```
/project
/desktop
/src
/components
/services
/production-engine
/export
/utils
/firebase
/rules
/functions
/config
/docs
README.md
```

41. Environment Configuration

- Development: Firebase development configuration, test data, development APIs
- Production: Firebase production configuration, production rules, production services

Never commit sensitive secrets into GitHub.

42. Testing Requirements

Frontend Tests

- Artwork upload
- Drag/drop
- Dimensions
- Quantity
- Contour generation

- Offset
- Smoothing
- Nesting
- Margins
- Spacing
- Rotation
- Registration marks
- Preview
- PDF export
- SVG export

Backend Tests

- Authentication
- User access
- Firestore rules
- Storage rules
- Project creation
- Project retrieval
- Project updates
- Unauthorized access prevention

43. Acceptance Test

Frontend:

```
PNG/JPG/SVG/PDF
↓
Sticker Size
↓
Quantity
↓
Contour
↓
Offset
↓
Media
↓
Spacing/Margins
↓
Nesting
↓
Registration Marks
↓
Preview
↓
PDF/SVG
```

Backend:

```
User
↓
Firebase Authentication
↓
```

```
User Profile
↓
Project
↓
Cloud Project Metadata
↓
Cloud Artwork Storage
```

44. 25-Day Development Schedule

Phase 1 — Frontend & Desktop Production Engine

- **Days 1–3:** React setup, desktop wrapper, application shell, UI architecture, GitHub setup
- **Days 4–6:** Drag/drop, PNG, JPG, SVG, PDF import, artwork preview
- **Days 7–10:** Sticker dimensions, quantity, contour generation, offset, smoothing
- **Days 11–14:** Sheet/roll, media dimensions, margins, spacing, rotation, nesting
- **Days 15–17:** Production layers, registration marks, Graphtec FC8600-oriented configuration
- **Days 18–20:** Live canvas, zoom, preview, production information, validation
- **Days 21–23:** PDF export, SVG export, physical dimensions, production output validation
- **Days 24–25:** Full QA, bug fixes, desktop packaging, final build, delivery

Phase 2 — Firebase Backend

- Firebase project configuration
- Authentication
- Firestore
- Storage
- Security Rules
- User profiles
- Project metadata
- Cloud project architecture
- Backend service layer
- Future licensing architecture
- Future subscription architecture

The exact duration of Phase 2 should be estimated separately if the client requests full cloud/licensing implementation.

45. Scope Boundary

Phase 1 Includes

- Frontend + Desktop + Local Production Engine
- Artwork import
- Sticker sizing
- Quantity
- Contour generation
- Offset

- Smoothing
- Media configuration
- Margins
- Spacing
- Rotation
- Nesting
- Registration marks
- Graphtec-oriented production
- Layers
- Preview
- PDF
- SVG
- Local project handling

Phase 2 Includes

- Firebase Authentication
- User profiles
- Firestore
- Storage
- Project cloud architecture
- Security Rules
- Backend service layer
- Future licensing/subscription foundation

Future Upgrade

- Monthly subscriptions
- Yearly subscriptions
- Payment gateway
- License activation
- Device management
- Subscription renewal
- Cloud dashboard
- Account management

46. Final Definition of Done

The project should ultimately deliver a standalone React-based desktop sticker-production application backed by Firebase infrastructure.

Primary production experience:

```
Drag Artwork → Set Size → Set Quantity → Generate Contour → Configure Media → Auto Nest →
Registration Marks → Preview → Export
```

The application must be structured so that the production engine works locally, while Firebase provides the foundation for future Accounts → Cloud Projects → Licensing → Monthly/Yearly Subscriptions → Payments.

This keeps the client's current requirement focused while ensuring the application can evolve into a commercial subscription product later.